KLE TECHNOLOGICAL UNIVERSITY, HUBLI

Department of Computer Science & Engineering (AI)

COURSE: AGENTIC AI (COURSE CODE: 26ECAC401)

InternTrack (AURA)

Autonomous University Regulatory Assistant — Master 14-Lab Implementation & Portal Contribution Report

Worklet Title:	Worklet 10 / Chapter 10: Internship Application & Tracking Portal
Team ID / Name:	InternTrack Team 11
Team Members:	• **Prerna A** (01FE23BCI037) • **Aishwarya B A** (01FE23BCI095) • **Sonu Vaishnavi** (01FE23BCI093) • **Nischita N** (01FE23BCI106)
Institution:	KLE Technological University, Hubli, Karnataka
Guiding Principle:	*"Agents draft and flag; a named human approves."*
Academic Term:	Academic Year 2026 — End-Semester Evaluation

MASTER TEST SUITE VERIFICATION STATUS

81 / 81 TESTS PASSED (100% PASS RATE)

All 14 Labs Fully Implemented, Unit-Tested, and Live on Production University Stack

Executive Architecture & Master Verification Summary

InternTrack (AURA) is an Agentic AI-driven academic internship lifecycle tracking and regulatory governance portal built specifically for university departments at **KLE Technological University, Hubli**. Constructed following the **Course 26ECAC401: Agentic AI 15 Worklets / 14-Lab Build Book (Chapter 10)**, it demonstrates how 14 specialized agentic capabilities work together to solve real-world internship challenges.

Master 14-Lab Verification Runner Matrix (81 of 81 Tests Passed)

Lab Number & Title	Phase	Core Agentic Capability	Test Suite Module	Tests	Pass Status
Lab 1: Agent vs Chatbot	Phase A	Goal-driven Clarify-Plan-Revise cycle	`tests/test_acceptance.py`	13 Tests (Suite)	**100% PASS**
Lab 2: Tool-Using Agent	Phase A	ReAct pattern with SQLite schema lookups	`tests/test_acceptance.py`	Integrated	**100% PASS**
Lab 3: Skill Creation	Phase A	Typed capability units (Plan/Match/Validate/Doc)	`tests/test_acceptance.py`	Integrated	**100% PASS**
Lab 4: Memory & Retrieval	Phase A	Dual SessionState & SQLite persistent archive	`tests/test_acceptance.py`	Integrated	**100% PASS**
Lab 5: MCP-Style Connector	Phase A	Bearer token authenticated & scoped connector	`tests/test_acceptance.py`	Integrated	**100% PASS**
Lab 6: Controlled Runtime	Phase A	Step budget limits & Coordinator Human Gate	`tests/test_acceptance.py`	Integrated	**100% PASS**
Lab 7: LangGraph Workflow	Phase B	StateGraph, conditional retry & checkpointing	`tests/test_acceptance.py`	Integrated	**100% PASS**
Lab 8: Parallel Swarm	Phase B	Fan-Out / Fan-In Reducer & Human Gate	`test_lab8_complete_suite.py`	13 Tests	**100% PASS**
Lab 9: Deep Research & Grounding	Phase C	Evidence Table Node & Adversarial Defense	`test_lab9_grounding.py`	9 Tests	**100% PASS**
GRAND TOTAL VERIFICATION SUITE				**81 Tests**	**81 / 81 PASSED (100% PASS RATE)**

Lab Number & Title	Phase	Core Agentic Capability	Test Suite Module	Tests	Pass Status
Lab 10: Ops Resilience & Self-Healing	Phase C	Diagnosis, Jitter Retry, Circuit Breaker, DLQ	`test_lab10_resilience.py`	9 Tests	**100% PASS**
Lab 11: Agentic SDLC Factory	Phase C	Spec Ingestion, Sandbox Harness, 100% Green Gate	`test_lab11_sdlc_factory.py`	9 Tests	**100% PASS**
Lab 12: Safety, Masking & Rollback	Phase D	RBAC Gate, Confidentiality Masking, Rollback	`test_lab12_governance.py`	9 Tests	**100% PASS**
Lab 13: Discipline Specialists	Phase D	5 Domain Packs (AI/ML, Systems, SQL, Java, DS)	`test_lab13_discipline_specialists.py`	9 Tests	**100% PASS**
Lab 14: Capstone Team-Owned System	Phase D	All Nodes Deployed + Faculty Evaluation Desk	`test_lab14_capstone.py`	10 Tests	**100% PASS**
GRAND TOTAL VERIFICATION SUITE				**81 Tests**	**81 / 81 PASSED (100% PASS RATE)**

Detailed Lab-by-Lab Contribution to the Internship Tracking Portal

Lab 1: Agent vs Chatbot (Agent Loop & Clarification Cycle)

Phase A: Foundations

> 🎯 **Specific Contribution to the Internship Portal:**
> Students frequently submit vague requests like "*I want an AI internship*" without specifying branch, CGPA, target cities, or company preferences. Instead of a dumb chatbot hallucinating a generic reply, Lab 1 provides an intelligent conversation loop (`aura/lab1_planner.py`). It identifies missing criteria, asks targeted clarifying questions, creates a structured roadmap, receives feedback, and refines the application plan.

EXPECTED INPUTS:
Natural language query, student USN, branch, CGPA.

EXPECTED OUTPUTS:
Clarifying question, structured plan, revised roadmap.

Code Implementation: `aura/lab1_planner.py` (Planner Agent, `clarify_intent`, `generate_plan`, `revise_plan`).

```
Test Output: test_acceptance.py -> test_06_lab1_planner_loop: PASSED (Clarify -> Plan -> Revise verifi
ed)
```

Lab 2: Tool-Using Agent (ReAct Pattern & Schema Lookups)

Phase A: Foundations

> 🎯 **Specific Contribution to the Internship Portal:**
> Students constantly inquire about real-time company cutoffs, stipend amounts, and available seats. Lab 2 connects the AI agent to the real SQLite database using the ReAct (Reasoning + Acting) loop (`aura/lab2_tools.py`). When asked "*Show companies in Bangalore with stipend > 30k*", the agent reasons, generates parameter-safe SQL queries against the `opportunities` table, observes real records, and provides accurate answers with zero hallucinations.

EXPECTED INPUTS:
Free-text natural language opportunity query.

EXPECTED OUTPUTS:
Thought-Action-Observation trace, verified real-world opportunity records.

Code Implementation: `aura/lab2_tools.py` (`query_database`, `search_opportunities`, `check_eligibility`).

```
Test Output: test_acceptance.py -> test_07_lab2_tool_agent: PASSED (ReAct tool execution with strict s
chema validation)
```

Lab 3: Skill Creation (Modular, Schema-Typed Capability Units)

Phase A: Foundations

> 🎯 **Specific Contribution to the Internship Portal:**
> Standardizes repetitive business logic into modular, reusable, type-safe skills (`aura/lab3_skills.py`) used by both the UI and AI agents: 1. `InternshipPlanSkill`: Generates semester milestone timelines. 2. `OpportunityMatchingSkill`: Calculates weighted matching scores based on student skills and company

requirements. 3. `ApplicationValidatorSkill`: Enforces university prerequisites (min CGPA, 0 active backlogs). 4. `DocumentFormatterSkill`: Produces formatted application summaries for review.

EXPECTED INPUTS:
Strictly typed dataclass/Pydantic schemas for student profile and preferences.

EXPECTED OUTPUTS:
Structured JSON response conforming to typed output schemas.

Code Implementation: `aura/lab3_skills.py` (4 typed skills conforming to schema invariants).

```
Test Output: test_acceptance.py -> test_08_lab3_skills: PASSED (All 4 modular skills pass schema valid
ation)
```

Lab 4: Memory & Retrieval (Dual-Tier Session Context & Historical Archive)

Phase A: Foundations

🎯 **Specific Contribution to the Internship Portal:**
When students re-apply upon rejection (creating version $v+1$) or interact across multiple days, the portal must remember past applications, coordinator feedback, and academic standing without starting from scratch. Lab 4 builds a dual-tier memory system (`aura/lab4_memory.py`): ephemeral working memory (`SessionState`) for the active conversation, and persistent SQLite retrieval for multi-version submission history and semantic opportunity matching.

EXPECTED INPUTS:
Student USN, active session turn messages, similarity search queries.

EXPECTED OUTPUTS:
Aggregated student context recalled across past sessions and submissions.

Code Implementation: `aura/lab4_memory.py` (`MemoryRetrievalEngine`, `SessionState`, `recall_student_context`).

```
Test Output: test_acceptance.py -> test_09_lab4_memory: PASSED (Multi-version history recalled; simila
rity search succeeds)
```

Lab 5: MCP-Style Connector (Bearer Token Auth & Role Scoping)

Phase A: Foundations

> 🎯 **Specific Contribution to the Internship Portal:**
> Enforces multi-tenant institutional privacy across the department. Students must **never** be able to view another student's private profile, submission history, or draft scores. Lab 5 implements a Model Context Protocol (MCP) inspired connector (`aura/lab5_connector.py`) using scoped Bearer tokens. Tokens carry `(actor_id, role)`: if a student queries another USN, the system throws an immediate `403 FORBIDDEN`; coordinators and HODs are granted wide departmental queue access.

EXPECTED INPUTS:
`Authorization: Bearer <token>`, caller actor ID, role (STUDENT, COORDINATOR, HOD).

EXPECTED OUTPUTS:
Scoped data response or `401 UNAUTHORIZED` / `403 FORBIDDEN` on scope breach.

Code Implementation: `aura/lab5_connector.py` (`GovernedMCPConnectorServer`, token issuance and validation).

```
Test Output: test_acceptance.py -> test_10_lab5_connector: PASSED (401 on missing auth, 403 on cross-student data reading)
```

Lab 6: Controlled Runtime & Governed Human Gate (Step Budgets & Interception)

Phase A: Foundations

> 🎯 **Specific Contribution to the Internship Portal:**
> Autonomous agents must never be allowed to run infinitely, nor should they autonomously confirm placement records or decrement company vacancies without human verification. Lab 6 creates an execution sandbox (`aura/lab6_runtime.py`) with strict step budgets (e.g. max 5 steps). Safe read-only checks execute automatically; consequential actions (committing placement records) are intercepted at `PAUSED_FOR_APPROVAL`, pausing until a named human **Internship Coordinator** approves or rejects with a mandatory reason.

EXPECTED INPUTS:
Student submission ID, step budget limit, consequential action payload.

EXPECTED OUTPUTS:
Execution pause at Human Gate; resumes only on named Coordinator sign-off.

Code Implementation: `aura/lab6_runtime.py` (`ControlledRuntimeEngine`, `RuntimeExecutionContainer`, human gate resume).

```
Test Output: test_acceptance.py -> test_11_lab6_runtime: PASSED (Safe step runs; risky step intercepted; Coordinator resumes)
```

Lab 7: LangGraph Workflow (Stateful Graph, Conditional Retries & Checkpoints)

Phase B: Orchestration

> 🎯 **Specific Contribution to the Internship Portal:**
> Coordinates multi-step application processing with deterministic state transitions: `Verify Eligibility -> Match Opportunities -> Rank Preferences -> Checkpoint Pause -> Human Signoff -> Commit Placement`. Lab 7 implements an official LangGraph `StateGraph` (`aura/lab7_langgraph.py`) with persistent memory

checkpoints (`MemorySaver`). If an applicant lacks prerequisites, the conditional router loops to secondary preferences; once ready, it suspends at the Human Gate, preventing any bypass.

EXPECTED INPUTS:
Student USN, 4 company preferences, LangGraph StateGraph state dict.

EXPECTED OUTPUTS:
StateGraph traversal through verify, match, rank, and pause at Coordinator checkpoint.

Code Implementation: `aura/lab7_langgraph.py` (Official LangGraph `StateGraph`, `MemorySaver`, conditional edges).

```
Test Output: test_acceptance.py -> test_12_lab7_langgraph: PASSED (StateGraph pauses at gate; bypass b
locked; approval commits)
```

Lab 8: Parallel Swarm (Multi-Preference Concurrent Evaluation & Reducer Merge)

Phase B: Orchestration

EXPECTED INPUTS:
4 prioritized student company preferences evaluated concurrently.

EXPECTED OUTPUTS:
4 parallel workers execute; fan-in reducer aggregates without race conditions; pauses at Coordinator gate.

🎯 **Specific Contribution to the Internship Portal:**
Students apply with up to 4 prioritized company choices. Checking each company sequentially causes slow response times. Lab 8 implements a parallel swarm (`aura/lab8_langgraph.py`) using `Annotated[List, operator.add]`. It dispatches 4 worker agents in parallel to check eligibility, stipend, and vacancies for Preferences 1, 2, 3, and 4 simultaneously. A fan-in reducer safely merges results, isolates worker failures, ranks the best matches, and presents the top recommendation to the Coordinator.

Code Implementation: `aura/lab8_langgraph.py` (Fan-out parallel workers, Pregel state reducer, Human Gate).

```
Test Output: test_lab8_complete_suite.py -> 13/13 Tests PASSED (Parallel execution, reducer merge, and
Human Gate signoff)
```

Lab 9: Deep Research & Grounding (Evidence Tables & Adversarial Defense)

Phase C: Applied Reliability

> 🎯 **Specific Contribution to the Internship Portal:**
> Prevents students or generative AI from inventing fake, unvetted companies (e.g. "*Quantum Apex Labs*") or non-existent corporate MoUs. Lab 9 implements an evidence-grounding engine (`aura/lab9_grounding.py`). Every candidate company is deeply researched against authoritative sources: faculty evaluation rubrics (SEC-1.1 to SEC-2.3), institutional student registries, and vetted opportunity catalogs. Hallucinated entities receive `in_scope=False` and are immediately rejected with `REJECTED_UNGROUNDED`.

EXPECTED INPUTS:
Candidate companies and claims, faculty rubric sections SEC-1.1 through SEC-2.3.

EXPECTED OUTPUTS:
Verified cited evidence table; adversarial ungrounded companies immediately marked `REJECTED_UNGROUNDED`.

Code Implementation: `aura/lab9_grounding.py` (`EvidenceRecord`, `EvidenceTable`, SHA-256 evidence hashing).

```
Test Output: test_lab9_grounding.py -> 9/9 Tests PASSED (Cited evidence verified; hallucinated compani
es rejected; 100% grounded)
```

Lab 10: Ops Resilience & Self-Healing (Circuit Breakers & Graceful Degradation)

Phase C: Applied Reliability

> 🎯 **Specific Contribution to the Internship Portal:**
> During peak college placement deadlines, external corporate job board APIs fail (HTTP 500) and SQLite database lock contentions occur. Lab 10 builds a self-healing operations engine (`aura/lab10_resilience.py`). It catches transient errors, applies exponential backoff with jitter (up to 3 retries), trips the circuit breaker to prevent cascading failures, falls back gracefully to a verified offline opportunity cache, and logs unresolvable errors to a Dead Letter Queue (DLQ).

EXPECTED INPUTS:
Simulated transient network failures, corporate API 500s, SQLite database busy locks.

EXPECTED OUTPUTS:
Automated fault diagnosis, 3 exponential backoff retries with jitter, graceful fallback to cached catalog, DLQ logging.

Code Implementation: `aura/lab10_resilience.py` (`ResilienceEngine`, `CircuitBreaker`, `DeadLetterQueue`).

```
Test Output: test_lab10_resilience.py -> 9/9 Tests PASSED (Diagnoses error, retries with backoff, circ
uit trips, fallback succeeds)
```

Lab 11: Agentic SDLC Factory (Automated Policy Translation & Green Gate)

Phase C: Applied Reliability

> 🎯 **Specific Contribution to the Internship Portal:**
> Departmental academic regulations change frequently (e.g. minimum CGPA raised from 7.5 to 8.0, or internship

duration updated from 3 to 4 months). Lab 11 implements an automated policy-to-code pipeline (`aura/lab11_sdlc_factory.py`). It ingests university policy specifications, hashes the requirement (`spec_hash`), generates candidate code patches, runs test suites in an isolated sandbox, and enforces the "Green Acceptance Gate": no code is deployed unless 100% of acceptance tests pass.

EXPECTED INPUTS:
Internship policy specification, test criteria (e.g. minimum CGPA 8.0, 4-month duration).

EXPECTED OUTPUTS:
Code patch generated, run against isolated sandbox test harness; 100% green gate signoff before deploy.

Code Implementation: `aura/lab11_sdlc_factory.py` (`AgenticSDLCFactory`, `GreenGateVerifier`, `SandboxExecutor`).

```
Test Output: test_lab11_sdlc_factory.py -> 9/9 Tests PASSED (Spec hashed, test harness runs, failing p
atch refined, green gate confirmed)
```

Lab 12: Safety & Governance (Approval Gate, Masking & 1-Click Rollback)

Phase D: Governance & Capstone

🎯 **Specific Contribution to the Internship Portal:**
Protects institutional privacy and provides fault tolerance during grading. Sensitive faculty viva remarks, employer NDA clauses, and placement risk metrics must remain masked from students. Lab 12 builds complete governance (`aura/lab12_governance.py`): enforces human approval gates before publishing release $v+1$ with SHA-256 version hashes, redacts sensitive fields for students with `[CONFIDENTIAL — REDACTED]`, and provides a 1-click reversible rollback that cleanly restores historical versions if human grading disputes arise.

EXPECTED INPUTS:
Candidate draft evaluation, human score overrides, caller actor role.

EXPECTED OUTPUTS:
Published version v+1 with SHA-256 hash; confidentiality masking for students; 1-click rollback to version v.

Code Implementation: `aura/lab12_governance.py` (`ROLE_PERMISSIONS`, `mask_confidential_content`, `publish_release`, `rollback_to_version`).

```
Test Output: test_lab12_governance.py -> 9/9 Tests PASSED (Student approval blocked, Coordinator publi
shes v+1, masking verified, rollback succeeds)
```

Lab 13: Discipline Specialists (Engineering Domain Invariants & Technical Rubrics)

Phase D: Governance & Capstone

> 🎯 **Specific Contribution to the Internship Portal:**
> Grading Computer Science / AI internship work requires verifying deep technical code deliverables, not just reading self-reported summaries. Lab 13 integrates 5 pluggable discipline specialist agents (`aura/lab13_discipline_specialists.py`): 1. *Python AI/ML Specialist:* Verifies PEP 484 type hints and PyTorch FP16 KV-cache tensor operations. 2. *C Systems / POSIX Specialist:* Enforces Valgrind memory cleanliness and thread-safe semaphores. 3. *SQL DBMS Specialist:* Validates 3NF schema design, covering B-Tree indexes, and transaction isolation. 4. *Java Enterprise Specialist:* Validates Spring Boot 3 IoC container design and `@Transactional` boundaries. 5. *Data Structures Specialist:* Proves algorithmic complexity ($O(N^2)$ Gale-Shapley stability).

EXPECTED INPUTS:
Student code deliverables in Python AI/ML, C Systems/POSIX, SQL DBMS, Java, or Data Structures.

EXPECTED OUTPUTS:
Deep invariant verification (PEP 484 type hints, Valgrind memory cleanliness, 3NF schema, Big-O proofs) and diagnostic score.

Code Implementation: `aura/lab13_discipline_specialists.py` (`DisciplineSpecialistManager`, 5 Specialist Domain Packs).

```
Test Output: test_lab13_discipline_specialists.py -> 9/9 Tests PASSED (All 5 domain packs verify clean deliverables and flag invariant bugs)
```

Lab 14: Capstone Project Studio (Team-Owned Integrated University Worklet)

Phase D: Governance & Capstone

> 🎯 **Specific Contribution to the Internship Portal:**
> Unifies all 14 individual lab capabilities into the complete, production-ready **InternTrack** portal. Connects the entire university lifecycle: student preference application -> AI candidate screening -> Coordinator approval gate -> ongoing 3–4 month progress tracking -> specialist technical evaluation -> HOD 12-credit sanction -> immutable audit trail. Demonstrated live at `http://localhost:8000/` with real-world college data: 65 students, 50 companies, 207 vacancies, and 44 active internships.

EXPECTED INPUTS:
End-to-end internship workflow request from applicant through final academic evaluation.

EXPECTED OUTPUTS:
Unified production system connecting connector, runtime, LangGraph, swarm, grounding, resilience, SDLC, governance, and specialists.

Code Implementation: `aura/lab14_capstone.py`, `server.py`, `static/js/app.js`, `static/js/workflow.js`, `static/js/aura_studio.js`.

```
Test Output: test_lab14_capstone.py -> 10/10 Tests PASSED (Complete multi-session capstone pipeline runs end to end with zero failures)
```

Actor Roles, Permissions & Academic Credit Clearance

InternTrack operates with three distinct personas designed to match university regulations and separation of duties at **KLE Technological University, Hubli**:

Actor Persona	Permissions Matrix	Confidentiality Level	Assigned Portal Responsibilities
Student `STUDENT`	• `can_approve: False` • `can_publish: False` • `can_rollback: False` • `can_view_confidential: False`	`PUBLIC_STUDENT` (Sensitive notes redacted)	• Automatically routes to **Internship Form** upon role selection. • Submits internship preference form (v1, v2...) or Placed-Company direct bypass. • Submits weekly/monthly progress journals and hours worked. • Views published grades & 12 credits (sensitive faculty notes masked).
Internship Coordinator `COORDINATOR`	• `can_approve: True` • `can_publish: True` • `can_rollback: True` • `can_view_confidential: True`	`RESTRICTED_FULL` (Unredacted clearance)	• Automatically routes to **Coordinator Review** upon role selection. • Screens student eligibility, CGPA thresholds, and company vacancies. • Approves applications (auto-decrements vacancies) or rejects with mandatory reason. • Enters rubric marks (out of 100) and sanctions 12 Academic Credits!
Head of Department `HOD`	• `can_approve: True` • `can_publish: True` • `can_rollback: True` • `can_view_confidential: True`	`RESTRICTED_FULL` (Apex Executive)	• Automatically routes to **HOD Approval Desk** upon role selection. • Grants official departmental sanction on approved applications. • Reviews cohort grading distribution and finalizes 12 Academic Credits signoff. • Oversees immutable audit logs and departmental accreditation metrics.

Dual Credit Sanction Feature (Coordinator & HOD)

In accordance with departmental academic guidelines at **KLE Technological University, Hubli**, **both the Internship Coordinator and the Head of Department (HOD)** possess authority to click **Finalize Signoff** to award the **12 Academic Credits** on the student's degree audit. When signed off, the system automatically transitions the internship to `COMPLETED`, sets evaluation status to `FINALIZED & CREDITED`, and logs the signatory's digital signature in the immutable append-only audit trail.

Production Database Cohort Verification

The system is seeded with an authentic, real-world cohort reflecting an entire engineering department:

- **Total Students in Registry:** 65 Students across Computer Science & Engineering (AI).
- **Published Corporate Opportunities:** 50 Opportunities across premier corporate partners (TCS, Microsoft, Google, AWS, Cisco, Infosys, Adobe, etc.).
- **Available Corporate Vacancies:** 207 Available Vacancies dynamically tracked in real-time.
- **Placed-Company Direct Confirmations:** 19 Students directly placed via campus placement bypass.
- **Active Tracked Internships & Progress Journals:** 44 Verified internship records with weekly work reports.

- **Milestone Evaluations:** 44 Evaluations with authentic, varied marks across Mentor (40), Presentation (30), and Report (30) rubric components.

InternTrack (AURA) — KLE Technological University, Hubli • Academic Year 2026
Verified against Course 26ECAC401: Agentic AI 15 Worklets / 14-Lab Build Book (Chapter 10)